

Live Session 3 — International Business Cycles

Indonesia and Asia-Pacific comparison

Caleb Tutty

Table of contents

Purpose	1
Data sources and reproducibility	1
1. Setup	2
2. Country set and data source	2
Download helper	3
Consumption and investment	7
3. Make the sample comparable	11
4. Construct the business-cycle component	12
4a. Build the HP filter manually	12
Step 1: take natural logs	13
Step 2: represent changes in trend growth with a matrix	13
Step 3: solve the first-order condition	13
Alternative: minimise the loss function directly	14
Why is $\lambda = 1600$?	16
4b. Apply the filter to every country and component	16
5. Indonesia: inspect the cycle	17
Growth-rate comparison	18
6. Statistical properties	19
Correlation with Indonesia's cycle	20
7. Rolling synchronisation	22
8. Export data for a D3 visualisation	24

Purpose

This notebook answers Question 2(c) from Live Session Problem Set 3 using quarterly real GDP, private consumption, and gross fixed capital formation. It treats each business-cycle component as the deviation of the logged real series from an HP-filtered trend and compares Indonesia with a set of Asia-Pacific economies plus the United States.

The workflow keeps the longest available quarterly history for each country and uses an unbalanced panel beginning in 1975. After taking logs and subtracting each country's trend, differences in currency scale do not affect the cycle measure.

Data sources and reproducibility

Every downloaded series comes from one of the following public sources. No API key is required.

- **Quarterly real GDP for the main country panel:** World Bank Global Economic Monitor (GEM), indicator `NYGDPMKTPSAKN` (real GDP in constant domestic currency, seasonally adjusted). The notebook retrieves it through the World Bank Indicators API.
- **Longer quarterly real GDP histories for Australia, South Korea, and the United States:** IMF *International Financial Statistics* (IFS), indicator `NGDP_R_SA_XDC`, distributed through the public DBnomics IMF/IFS dataset.
- **Private consumption:** IMF IFS real private-sector final consumption, indicators `NCP_R_SA_XDC` and, where the source-seasonally-adjusted series is unavailable, `NCP_R_NSA_XDC`, from the same DBnomics IMF/IFS dataset.
- **Investment:** IMF IFS real gross fixed capital formation, indicators `NFI_R_SA_XDC` and the non-seasonally-adjusted fallback `NFI_R_NSA_XDC`, from the same DBnomics IMF/IFS dataset.

The exact observations used by a normal render are preserved in the local GDP snapshot and consumption/investment snapshot. Their companion metadata files record when the snapshots were retrieved. Setting `REFRESH_DATA=true` deliberately replaces these files with the latest source observations.

1. Setup

```
required <- c(
  "tidyverse", "zoo", "mFilter", "moments", "scales", "jsonlite", "httr2"
)
missing <- required[!vapply(required, requireNamespace, logical(1), quietly
= TRUE)]
if (length(missing)) {
  stop(
    "Missing project dependencies: ",
    paste(missing, collapse = ", "),
    ". Run renv::restore() before rendering."
  )
}

library(tidyverse)
library(zoo)
library(mFilter)
library(moments)
library(scales)

theme_set(theme_minimal(base_size = 12))
```

2. Country set and data source

The preferred measure is **real GDP, quarterly, seasonally adjusted**. The World Bank *Global Economic Monitor* (GEM) supplies a consistent series for all countries. For Australia, South Korea, and the United States, longer compatible IMF *International Financial Statistics* series distributed by DBnomics extend the history through the requested 1975 boundary. The panel is intentionally unbalanced: countries enter when their quarterly series begins.

```
countries <- tribble(
  ~country,      ~iso2, ~iso3, ~source,      ~series_id,      ~group,
  "Indonesia",   "ID",  "IDN", "World Bank GEM", "NYGDPMKTPSAKN", "Focus",
  "Malaysia",    "MY",  "MYS", "World Bank GEM", "NYGDPMKTPSAKN", "ASEAN",
  "Thailand",     "TH",  "THA", "World Bank GEM", "NYGDPMKTPSAKN", "ASEAN",
  "Philippines", "PH",  "PHL", "World Bank GEM", "NYGDPMKTPSAKN", "ASEAN",
  "Singapore",   "SG",  "SGP", "World Bank GEM", "NYGDPMKTPSAKN", "ASEAN",
  "Vietnam",     "VN",  "VNM", "World Bank GEM", "NYGDPMKTPSAKN", "ASEAN",
  "Australia",   "AU",  "AUS", "IMF IFS via DBnomics", "Q.AU.NGDP_R_SA_XDC",
  "Asia-Pacific",
  "New Zealand", "NZ",  "NZL", "World Bank GEM", "NYGDPMKTPSAKN", "Asia-
Pacific",
  "Japan",       "JP",  "JPN", "World Bank GEM", "NYGDPMKTPSAKN",
  "Asia-Pacific",
  "South Korea", "KR",  "KOR", "IMF IFS via DBnomics", "Q.KR.NGDP_R_SA_XDC",
  "Asia-Pacific",
  "India",       "IN",  "IND", "World Bank GEM", "NYGDPMKTPSAKN",
  "Asia-Pacific",
  "China",       "CN",  "CHN", "World Bank GEM", "NYGDPMKTPSAKN",
  "Asia-Pacific",
  "United States", "US",  "USA", "IMF IFS via DBnomics", "Q.US.NGDP_R_SA_XDC",
  "Comparator"
)

countries
```

```
# A tibble: 13 × 6
  country      iso2 iso3 source      series_id      group
  <chr>      <chr> <chr> <chr>      <chr>      <chr>
1 Indonesia   ID    IDN World Bank GEM NYGDPMKTPSAKN Focus
2 Malaysia    MY    MYS World Bank GEM NYGDPMKTPSAKN ASEAN
3 Thailand     TH    THA World Bank GEM NYGDPMKTPSAKN ASEAN
4 Philippines  PH    PHL World Bank GEM NYGDPMKTPSAKN ASEAN
5 Singapore    SG    SGP World Bank GEM NYGDPMKTPSAKN ASEAN
6 Vietnam     VN    VNM World Bank GEM NYGDPMKTPSAKN ASEAN
7 Australia   AU    AUS IMF IFS via DBnomics Q.AU.NGDP_R_SA_XDC Asia-Pacif...
8 New Zealand  NZ    NZL World Bank GEM NYGDPMKTPSAKN Asia-Pacif...
9 Japan       JP    JPN World Bank GEM NYGDPMKTPSAKN Asia-Pacif...
10 South Korea KR    KOR IMF IFS via DBnomics Q.KR.NGDP_R_SA_XDC Asia-Pacif...
11 India      IN    IND World Bank GEM NYGDPMKTPSAKN Asia-Pacif...
12 China      CN    CHN World Bank GEM NYGDPMKTPSAKN Asia-Pacif...
13 United States US    USA IMF IFS via DBnomics Q.US.NGDP_R_SA_XDC Comparator
```

Download helper

The World Bank and DBnomics endpoints are public and require no API key. Downloads use the R package `httr2`. The first render saves a raw-data snapshot under `data/raw/`; later

renders reuse it so source revisions do not silently change the results. Set `REFRESH_DATA=true` to download and replace the snapshot deliberately.

```
download_file <- function(url, extension) {
  destination <- tempfile(fileext = extension)

  tryCatch({
    httr2::request(url) %>%
      httr2::req_user_agent("LS3-business-cycles/1.0") %>%
      httr2::req_timeout(seconds = 60) %>%
      httr2::req_retry(max_tries = 3) %>%
      httr2::req_perform(path = destination)

    if (!file.exists(destination) || file.size(destination) == 0) {
      stop("The server returned an empty response.")
    }

    destination
  }, error = function(e) {
    unlink(destination)
    warning("Download failed: ", conditionMessage(e), call. = FALSE)
    NULL
  })
}

read_ifs_gdp <- function(country_map) {
  country_mask <- paste(country_map$iso2, collapse = "+")
  url <- paste0(
    "https://api.db.nomics.world/v22/series/IMF/IFS/Q.",
    country_mask, ".NGDP_R_SA_XDC?observations=1"
  )
  destination <- download_file(url, ".json")
  on.exit(if (!is.null(destination)) unlink(destination), add = TRUE)

  if (is.null(destination)) {
    stop("Could not download quarterly real GDP from IMF IFS via DBnomics.")
  }

  payload <- jsonlite::fromJSON(destination, simplifyVector = FALSE)
  docs <- payload$series$docs
  if (is.null(docs) || !length(docs)) {
    stop("DBnomics returned no IMF IFS GDP series.")
  }

  map_dfr(docs, function(doc) {
    values <- vapply(
      doc$value,
```

```

    function(value) if (is.null(value)) NA_real_ else as.numeric(value),
    numeric(1)
  )

  tibble(
    iso2 = doc$dimensions$REF_AREA,
    series_id = doc$series_code,
    date = as.Date(unlist(doc$period_start_day)),
    real_gdp = values
  )
}) %>%
  filter(!is.na(real_gdp), real_gdp > 0) %>%
  inner_join(
    country_map %>% select(country, iso2, iso3, group, source),
    by = "iso2"
  ) %>%
  select(country, iso2, iso3, group, source, series_id, date, real_gdp)
}

read_gem <- function(country_map) {
  country_codes <- paste(country_map$iso3, collapse = ";")
  end_period <- paste0(format(Sys.Date(), "%Y"), "Q4")
  url <- paste0(
    "https://api.worldbank.org/v2/country/", country_codes,
    "/indicator/NYGDPMKTPSAKN?source=15",
    "&date=1975Q1:", end_period,
    "&format=json&per_page=20000"
  )
  destination <- download_file(url, ".json")
  on.exit(if (!is.null(destination)) unlink(destination), add = TRUE)

  if (is.null(destination)) {
    stop("Could not download World Bank GEM quarterly GDP.")
  }

  payload <- jsonlite::fromJSON(destination, simplifyDataFrame = TRUE)
  if (length(payload) < 2 || !is.data.frame(payload[[2]])) {
    stop("World Bank GEM returned an unexpected response.")
  }

  records <- payload[[2]]
  quarter_number <- as.integer(substr(records$date, 6, 6))
  quarter_month <- c("01", "04", "07", "10")[quarter_number]

  tibble(
    iso3 = records$country$id,
    date = as.Date(paste0(substr(records$date, 1, 4), "-", quarter_month,

```

```

"-01")),
  real_gdp = as.numeric(records$value)
) %>%
  filter(!is.na(real_gdp), real_gdp > 0) %>%
  inner_join(country_map, by = "iso3") %>%
  select(country, iso2, iso3, group, source, series_id, date, real_gdp)
}

raw_dir <- file.path("data", "raw")
raw_data_path <- file.path(raw_dir, "quarterly_real_gdp.csv")
raw_metadata_path <- file.path(raw_dir, "quarterly_real_gdp_metadata.csv")
refresh_data <- tolower(Sys.getenv("REFRESH_DATA", "false")) %in% c("1",
"true", "yes")

if (refresh_data || !file.exists(raw_data_path)) {
  ifs_countries <- countries %>% filter(source == "IMF IFS via DBnomics")
  gem_countries <- countries %>% filter(source == "World Bank GEM")
  gdp_raw <- bind_rows(
    read_ifs_gdp(ifs_countries),
    read_gem(gem_countries)
  )

  if (nrow(gdp_raw) == 0) {
    stop("No GDP observations were downloaded; the raw-data snapshot was
not updated.")
  }

  absent <- setdiff(countries$country, unique(gdp_raw$country))
  if (length(absent)) {
    stop("No quarterly GDP data returned for: ", paste(absent, collapse =
", "))
  }

  dir.create(raw_dir, recursive = TRUE, showWarnings = FALSE)
  write_csv(gdp_raw, raw_data_path)
  write_csv(
    tibble(
      retrieved_at_utc = format(Sys.time(), tz = "UTC", usetz = TRUE),
      sources = paste(sort(unique(gdp_raw$source)), collapse = "; "),
      country_count = n_distinct(gdp_raw$country),
      observation_count = nrow(gdp_raw)
    ),
    raw_metadata_path
  )
} else {
  gdp_raw <- read_csv(
    raw_data_path,

```

```

col_types = cols(
  country = col_character(),
  iso2 = col_character(),
  iso3 = col_character(),
  group = col_character(),
  source = col_character(),
  series_id = col_character(),
  date = col_date(),
  real_gdp = col_double()
)
}

coverage <- gdp_raw %>%
  group_by(country, source, series_id) %>%
  summarise(first = min(date), last = max(date), n = n(), .groups = "drop")

coverage

```

```

# A tibble: 13 × 6
  country      source      series_id      first      last      n
  <chr>        <chr>        <chr>        <date>    <date>    <int>
1 Australia   IMF IFS via DBnomics Q.AU.NGDP_R_S... 1959-07-01 2025-01-01 263
2 China       World Bank GEM      NYGDPMKTPSAKN 1991-01-01 2025-10-01 140
3 India       World Bank GEM      NYGDPMKTPSAKN 1996-04-01 2025-10-01 119
4 Indonesia   World Bank GEM      NYGDPMKTPSAKN 1987-01-01 2025-10-01 156
5 Japan       World Bank GEM      NYGDPMKTPSAKN 1987-01-01 2025-10-01 156
6 Malaysia    World Bank GEM      NYGDPMKTPSAKN 2005-01-01 2025-10-01 84
7 New Zealand World Bank GEM      NYGDPMKTPSAKN 1987-04-01 2025-07-01 154
8 Philippines World Bank GEM      NYGDPMKTPSAKN 1987-01-01 2025-10-01 156
9 Singapore   World Bank GEM      NYGDPMKTPSAKN 1987-01-01 2025-10-01 156
10 South Korea IMF IFS via DBnomics Q.KR.NGDP_R_S... 1960-01-01 2025-04-01 262
11 Thailand    World Bank GEM      NYGDPMKTPSAKN 1993-01-01 2025-10-01 132
12 United States IMF IFS via DBnomics Q.US.NGDP_R_S... 1950-01-01 2025-04-01 302
13 Vietnam     World Bank GEM      NYGDPMKTPSAKN 1990-10-01 2025-10-01 141

```

Consumption and investment

My choice of consumption and investment datasets come from IMF *International Financial Statistics* through the public DBnomics API. Source-seasonally- adjusted series are preferred; when unavailable, a real non-seasonally-adjusted series is adjusted with a periodic STL (Seasonal and Trend decomposition using LOESS) decomposition. The IMF doesn't publish this data for all countries (e.g. Vietnam).

```

component_definitions <- tribble(
  ~component,    ~sa_indicator,    ~nsa_indicator,
  "Consumption", "NCP_R_SA_XDC",  "NCP_R_NSA_XDC",
  "Investment",  "NFI_R_SA_XDC",  "NFI_R_NSA_XDC"
)

extract_dbnomics_components <- function(country_map) {
  country_mask <- paste(country_map$iso2, collapse = "+")
  indicator_mask <- paste(
    c(component_definitions$sa_indicator,
component_definitions$nsa_indicator),
    collapse = "+"
  )
  url <- paste0(
    "https://api.db.nomics.world/v22/series/IMF/IFS/Q.",
    country_mask, ".", indicator_mask, "?observations=1"
  )
  destination <- download_file(url, ".json")
  on.exit(if (!is.null(destination)) unlink(destination), add = TRUE)

  if (is.null(destination)) {
    stop("Could not download IMF IFS components from DBnomics.")
  }

  payload <- jsonlite::fromJSON(destination, simplifyVector = FALSE)
  docs <- payload$series$docs
  if (is.null(docs) || !length(docs)) {
    stop("DBnomics returned no IMF IFS component series.")
  }

  candidates <- map_dfr(docs, function(doc) {
    indicator <- doc$dimensions$INDICATOR
    values <- vapply(
      doc$value,
      function(value) if (is.null(value)) NA_real_ else as.numeric(value),
      numeric(1)
    )

    tibble(
      iso2 = doc$dimensions$REF_AREA,
      component = if_else(str_starts(indicator, "NCP_"), "Consumption",
"Investment"),
      series_id = doc$series_code,
      source_adjustment = if_else(str_detect(indicator, "_SA_"), "SA", "NSA"),
      date = as.Date(unlist(doc$period_start_day)),
      value = values
    )
  })

```

```

}) %>%
  filter(!is.na(value), value > 0) %>%
  inner_join(
    country_map %>% select(country, iso2, iso3, group),
    by = "iso2"
  )

selected_series <- candidates %>%
  distinct(country, component, series_id, source_adjustment) %>%
  mutate(priority = if_else(source_adjustment == "SA", 1L, 2L)) %>%
  group_by(country, component) %>%
  slice_min(priority, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  select(country, component, series_id)

candidates %>%
  semi_join(selected_series, by = c("country", "component", "series_id"))
}

adjust_component_seasonality <- function(df) {
  df <- arrange(df, date)
  if (first(df$source_adjustment) == "SA") {
    return(mutate(
      df,
      real_value = value,
      adjustment_method = "Source seasonally adjusted"
    ))
  }
}

expected_dates <- seq(min(df$date), max(df$date), by = "3 months")
if (nrow(df) < 20 || !identical(df$date, expected_dates)) {
  warning("Cannot seasonally adjust incomplete series: ",
first(df$series_id))
  return(tibble())
}

start_year <- as.integer(format(first(df$date), "%Y"))
start_quarter <- (as.integer(format(first(df$date), "%m")) - 1L) %/% 3L
+ 1L
fit <- stl(
  ts(log(df$value), start = c(start_year, start_quarter), frequency = 4),
  s.window = "periodic",
  robust = TRUE
)

mutate(
  df,

```

```

    real_value = exp(log(value) - as.numeric(fit$time.series[, "seasonal"])),
    adjustment_method = "Periodic STL adjustment of source NSA series"
  )
}

component_data_path <- file.path(raw_dir, "quarterly_real_components.csv")
component_metadata_path <-
file.path(raw_dir, "quarterly_real_components_metadata.csv")

if (refresh_data || !file.exists(component_data_path)) {
  components_raw <- extract_dbnomics_components(countries) %>%
    group_by(country, component) %>%
    group_modify(~ adjust_component_seasonality(.x)) %>%
    ungroup() %>%
    mutate(source = "IMF IFS via DBnomics") %>%
    select(
      country, iso2, iso3, group, component, source, series_id,
      source_adjustment, adjustment_method, date, real_value
    )

  dir.create(raw_dir, recursive = TRUE, showWarnings = FALSE)
  write_csv(components_raw, component_data_path)
  write_csv(
    tibble(
      retrieved_at_utc = format(Sys.time(), tz = "UTC", usetz = TRUE),
      source = "IMF International Financial Statistics via DBnomics",
      country_count = n_distinct(components_raw$country),
      series_count = n_distinct(components_raw$series_id),
      observation_count = nrow(components_raw)
    ),
    component_metadata_path
  )
} else {
  components_raw <- read_csv(
    component_data_path,
    col_types = cols(
      country = col_character(),
      iso2 = col_character(),
      iso3 = col_character(),
      group = col_character(),
      component = col_character(),
      source = col_character(),
      series_id = col_character(),
      source_adjustment = col_character(),
      adjustment_method = col_character(),
      date = col_date(),
      real_value = col_double()
    )
  )
}

```

```

    )
  )
}

component_coverage <- crossing(
  country = countries$country,
  component = component_definitions$component
) %>%
  left_join(
    components_raw %>%
      group_by(country, component, adjustment_method) %>%
      summarise(first = min(date), last = max(date), n = n(), .groups = "drop"),
    by = c("country", "component")
  ) %>%
  arrange(country, component)

component_coverage

```

```

# A tibble: 26 × 6
  country component adjustment_method first last n
  <chr>    <chr>      <chr>      <date> <date> <int>
1 Australia Consumption Source seasonally adjusted 1959-07-01 2025-01-01 263
2 Australia Investment Source seasonally adjusted 1959-07-01 2025-01-01 263
3 China    Consumption <NA>          NA      NA      NA
4 China    Investment  <NA>          NA      NA      NA
5 India    Consumption Source seasonally adjusted 1996-04-01 2025-01-01 116
6 India    Investment Source seasonally adjusted 1996-04-01 2025-01-01 116
7 Indonesia Consumption Source seasonally adjusted 2000-01-01 2025-01-01 101
8 Indonesia Investment Source seasonally adjusted 2000-01-01 2025-01-01 101
9 Japan    Consumption Source seasonally adjusted 1994-01-01 2025-01-01 125
10 Japan   Investment Source seasonally adjusted 1994-01-01 2025-01-01 125
# i 16 more rows

```

3. Make the sample comparable

The analysis window begins in **1975 Q1**, but each country enters when its quarterly series becomes available. Summary statistics use every observation available for that country; correlations use overlapping observations only.

```

start_date <- as.Date("1975-01-01")
end_date <- max(gdp_raw$date)

series_data <- bind_rows(
  gdp_raw %>%
    transmute(
      country, iso2, iso3, group, component = "GDP", source, series_id,

```

```

      adjustment_method = "Source seasonally adjusted", date,
      value = real_gdp
    ),
    components_raw %>%
      transmute(
        country, iso2, iso3, group, component, source, series_id,
        adjustment_method, date, value = real_value
      )
    ) %>%
    filter(date >= start_date, date <= end_date) %>%
    arrange(component, country, date)

```

4. Construct the business-cycle component

For quarterly data, the conventional Hodrick–Prescott smoothing parameter is $\lambda = 1600$. We work with the log of each real expenditure series:

$$\text{cycle}_{ijt} = 100[\log(X_{ijt}) - \text{trend}_{ijt}],$$

where j denotes GDP, private consumption, or gross fixed capital formation. The result is approximately the percentage deviation from the component’s estimated trend.

4a. Build the HP filter manually

There is an R package called `mFilter` which includes an `hpfilter` function. I think the goal of this exercise is to understand in a bit more detail how the math works, so this section walks through building it manually.

The HP filter chooses a trend τ_t that balances fit against smoothness. For a logged series $y_t = \log(X_t)$, the notation means:

- X_t is the observed real GDP, consumption, or investment level in quarter t .
- $y_t = \log(X_t)$ is that observed level after taking its natural logarithm. It is the input to the filter, not something chosen by the filter.
- τ_t is the unobserved smooth trend in the same log units as y_t . It is the quantity chosen by the HP calculation.
- $c_t = y_t - \tau_t$ is the estimated cyclical gap. Multiplying it by 100 gives the approximate percentage deviation of the observed series from trend.

The filter chooses the complete sequence $\tau_1, \dots, \tau_T$ by solving

$$\min_{\tau_1, \dots, \tau_T} \left\{ \sum_{t=1}^T (y_t - \tau_t)^2 + \lambda \sum_{t=2}^{T-1} [(\tau_{t+1} - \tau_t) - (\tau_t - \tau_{t-1})]^2 \right\}.$$

The first sum penalises a trend that is far from the data. The expression in the second sum is the change in the trend’s growth rate—its second difference— so that term penalises a trend that bends sharply.

Step 1: take natural logs

Logging makes proportional changes comparable across countries and units. A log gap of 0.01 is approximately a one-percent gap, which is why the final cycle is multiplied by 100.

```
lambda_hp <- 1600

indonesia_example <- series_data %>%
  filter(country == "Indonesia", component == "GDP") %>%
  arrange(date) %>%
  transmute(date, real_level = value, log_value = log(value))

indonesia_example %>% slice_head(n = 8)
```

```
# A tibble: 8 × 3
  date      real_level log_value
<date>      <dbl>     <dbl>
1 1987-01-01 508458716.     20.0
2 1987-04-01 534129198.     20.1
3 1987-07-01 532482757.     20.1
4 1987-10-01 531655632.     20.1
5 1988-01-01 532437460.     20.1
6 1988-04-01 571378035.     20.2
7 1988-07-01 567219214.     20.2
8 1988-10-01 583226308.     20.2
```

Note: this is Indonesian Rupiah in 2010 Rupiah values.

Step 2: represent changes in trend growth with a matrix

Let $\mathbf{y}$ and $\boldsymbol{\tau}$ contain all T observations. A second-difference matrix D has 1, -2 , 1 in each successive row, so $D\boldsymbol{\tau}$ contains $\tau_{t-1} - 2\tau_t + \tau_{t+1}$. Here is the leading corner of that matrix:

```
n_periods <- nrow(indonesia_example)
D2 <- diff(diag(n_periods), differences = 2)
D2[1:4, 1:6]
```

```
      [,1] [,2] [,3] [,4] [,5] [,6]
[1,]    1   -2    1    0    0    0
[2,]    0    1   -2    1    0    0
[3,]    0    0    1   -2    1    0
[4,]    0    0    0    1   -2    1
```

The objective can therefore be written compactly as

$$(\mathbf{y} - \boldsymbol{\tau})'(\mathbf{y} - \boldsymbol{\tau}) + \lambda(D\boldsymbol{\tau})'(D\boldsymbol{\tau}).$$

Step 3: solve the first-order condition

Differentiating with respect to $\boldsymbol{\tau}$ and setting the result to zero gives

$$(I_T + \lambda D' D) \tau = y, \quad \hat{\tau} = (I_T + \lambda D' D)^{-1} y.$$

In code, `solve(A, y)` solves $A\tau = y$ directly.

```
hp_manual <- function(x, lambda = 1600) {
  x <- as.numeric(x)
  n <- length(x)
  if (n < 3 || any(!is.finite(x))) {
    stop("The manual HP filter needs at least three finite observations.")
  }

  D2 <- diff(diag(n), differences = 2)
  system_matrix <- diag(n) + lambda * crossprod(D2)
  trend <- as.numeric(solve(system_matrix, x))

  tibble(
    hp_trend_manual = trend,
    cycle_manual = 100 * (x - trend)
  )
}

manual_indonesia <- bind_cols(
  indonesia_example,
  hp_manual(indonesia_example$log_value, lambda = lambda_hp)
)

manual_indonesia %>%
  select(date, real_level, log_value, hp_trend_manual, cycle_manual) %>%
  slice_head(n = 8)
```

```
# A tibble: 8 × 5
  date      real_level log_value hp_trend_manual cycle_manual
<date>      <dbl>    <dbl>         <dbl>         <dbl>
1 1987-01-01 508458716.    20.0          20.0          0.0826
2 1987-04-01 534129198.    20.1          20.1           2.98
3 1987-07-01 532482757.    20.1          20.1           0.634
4 1987-10-01 531655632.    20.1          20.1          -1.56
5 1988-01-01 532437460.    20.1          20.1          -3.45
6 1988-04-01 571378035.    20.2          20.1           1.57
7 1988-07-01 567219214.    20.2          20.2          -1.21
8 1988-10-01 583226308    20.2          20.2          -0.486
```

Alternative: minimise the loss function directly

The matrix equation is not the only way to compute the answer. We can code the two terms of the loss function exactly as written and let a numerical optimiser search over the T possible trend values. The gradient below distributes the effect of each second difference back to the three trend observations involved in it; it does not construct D or solve a matrix equation.

```

hp_loss <- function(tau, y, lambda) {
  fit_loss <- sum((y - tau)^2)
  smoothness_loss <- lambda * sum(diff(tau, differences = 2)^2)
  fit_loss + smoothness_loss
}

hp_loss_gradient <- function(tau, y, lambda) {
  gradient <- 2 * (tau - y)
  curvature <- diff(tau, differences = 2)

  for (i in seq_along(curvature)) {
    gradient[i] <- gradient[i] + 2 * lambda * curvature[i]
    gradient[i + 1] <- gradient[i + 1] - 4 * lambda * curvature[i]
    gradient[i + 2] <- gradient[i + 2] + 2 * lambda * curvature[i]
  }

  gradient
}

hp_optim <- function(x, lambda = 1600) {
  result <- optim(
    par = x,
    fn = hp_loss,
    gr = hp_loss_gradient,
    y = x,
    lambda = lambda,
    method = "BFGS",
    control = list(maxit = 5000, reltol = 1e-12)
  )

  list(
    trend = result$par,
    cycle = 100 * (x - result$par),
    convergence = result$convergence,
    objective = result$value
  )
}

optim_indonesia <- hp_optim(
  indonesia_example$log_value,
  lambda = lambda_hp
)

tibble(
  convergence_code = optim_indonesia$convergence,
  minimised_loss = optim_indonesia$objective,
  maximum_cycle_difference_from_matrix = max(

```

```

    abs(optim_indonesia$cycle - manual_indonesia$cycle_manual)
  )
)

```

```

# A tibble: 1 × 3
  convergence_code minimised_loss maximum_cycle_difference_from_matrix
      <int>          <dbl>                <dbl>
1           0          0.146                0.000000161

```

method = "BFGS" asks `optim()` to use the **Broyden–Fletcher–Goldfarb–Shanno** algorithm.

Why is $\lambda = 1600$?

λ controls the trade-off rather than describing the data itself:

- At $\lambda = 0$, the fitted trend follows every observation and the cycle is zero.
- As λ increases, changes in trend growth become more costly, the trend becomes smoother, and more medium-frequency movement is assigned to the cycle.
- As $\lambda \rightarrow \infty$, the fitted trend approaches a straight line.

The value 1600 is the convention proposed by Hodrick and Prescott (1997) for quarterly macroeconomic data; it is not estimated from this sample and is not universally optimal. Sampling frequency matters because a second difference has a different scale when observations are closer together. Ravn and Uhlig (2002) recommend scaling λ by the fourth power of the frequency ratio. Taking quarterly 1600 as the benchmark gives $1600/4^4 = 6.25$ for annual data and $1600 \times 3^4 = 129,600$ for monthly data. We retain 1600 here because every series entering the filter is quarterly, and later treat the choice as a modelling assumption rather than a fact.

4b. Apply the filter to every country and component

After explaining the mechanics we can just use the package version `mFilter::hpfilter()`.

```

hp_one_series <- function(df) {
  df <- arrange(df, date)
  if (nrow(df) < 20) return(tibble())

  x <- log(df$value)
  hp <- mFilter::hpfilter(x, freq = lambda_hp, type = "lambda")

  df %>%
    mutate(
      log_value = x,
      hp_trend = as.numeric(hp$trend),
      cycle = 100 * as.numeric(hp$cycle),
      qoq_growth = 100 * (log_value - lag(log_value)),
      qoq_annualised = 400 * (log_value - lag(log_value)),
      yoy_growth = 100 * (log_value - lag(log_value, 4))
    )
}

```

```

}

cycles <- series_data %>%
  group_by(country, component) %>%
  group_modify(~ hp_one_series(.x)) %>%
  ungroup()

manual_package_check <- cycles %>%
  filter(country == "Indonesia", component == "GDP") %>%
  select(date, package_cycle = cycle) %>%
  inner_join(
    manual_indonesia %>% select(date, cycle_manual),
    by = "date"
  ) %>%
  summarise(
    maximum_absolute_difference_percentage_points =
      max(abs(package_cycle - cycle_manual))
  )

manual_package_check

```

```

# A tibble: 1 × 1
  maximum_absolute_difference_percentage_points
              <dbl>
1                4.83e-10

```

The reported maximum difference should be effectively zero (up to floating- point rounding), demonstrating that the manual linear-system solution and `mFilter` implement the same HP calculation.

5. Indonesia: inspect the cycle

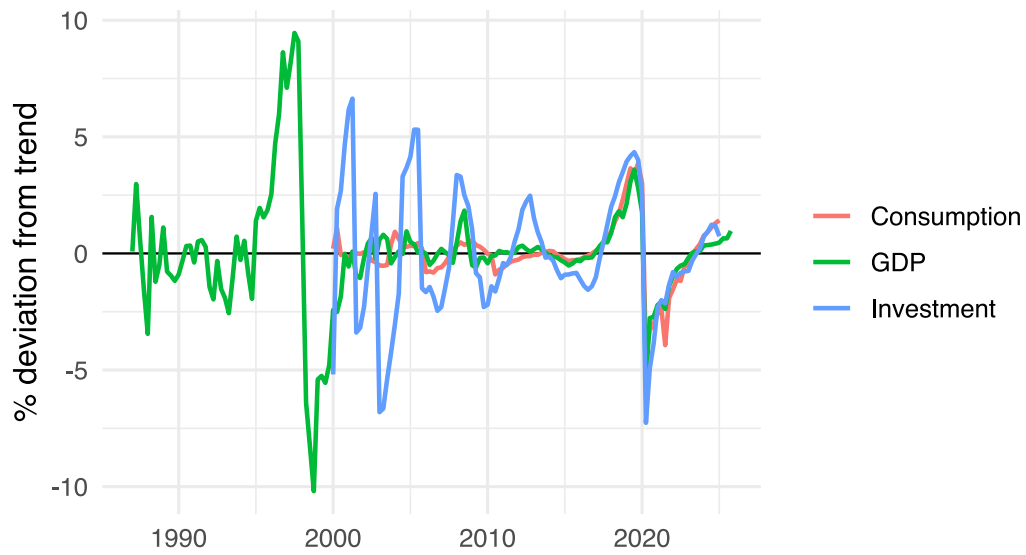
```

cycles %>%
  filter(country == "Indonesia") %>%
  ggplot(aes(date, cycle, colour = component)) +
  geom_hline(yintercept = 0, linewidth = 0.4) +
  geom_line(linewidth = 0.8) +
  labs(
    title = "Indonesia: output and expenditure cycles",
    subtitle = "HP-filtered log real series,  $\lambda = 1600$ ",
    x = NULL,
    y = "% deviation from trend",
    colour = NULL
  )

```

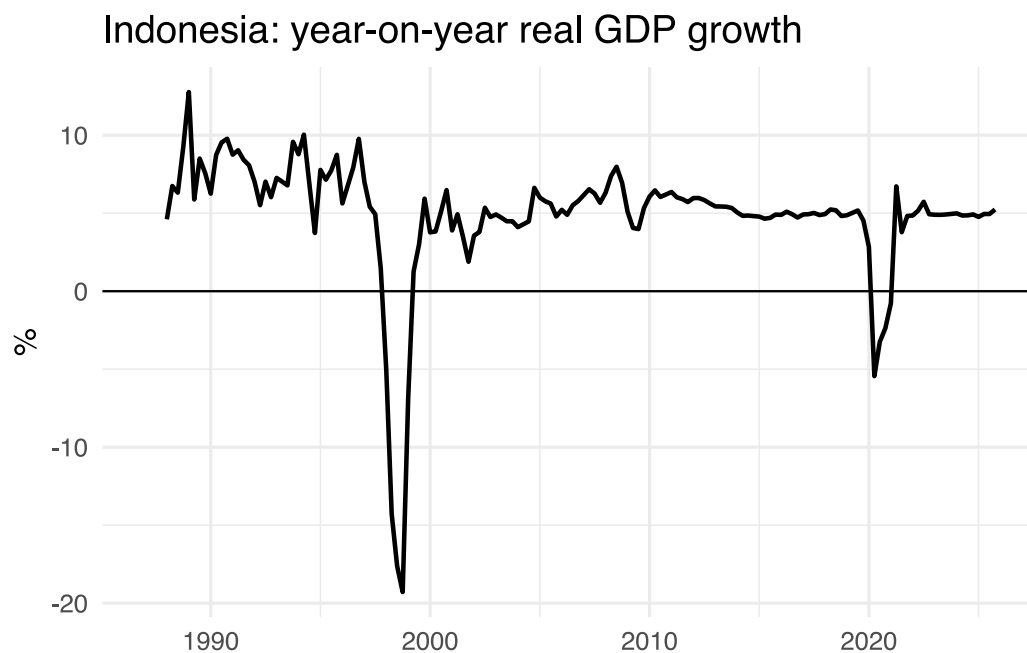
Indonesia: output and expenditure cycles

HP-filtered log real series, $\lambda = 1600$



Growth-rate comparison

```
cycles %>%
  filter(country == "Indonesia", component == "GDP") %>%
  ggplot(aes(date, yoy_growth)) +
  geom_hline(yintercept = 0, linewidth = 0.4) +
  geom_line(linewidth = 0.8) +
  labs(
    title = "Indonesia: year-on-year real GDP growth",
    x = NULL,
    y = "%"
  )
)
```



Note 1998 saw the Asian Financial Crisis as well as the crisis which led to the fall of the dictator Suharto.

6. Statistical properties

The main statistics are volatility, persistence, and co-movement. This shows international business-cycle patterns: consumption is often smoother than output, while investment is usually more volatile. Skewness and kurtosis moments as also added.

```
safe_cor_lag1 <- function(x) {
  x <- x[is.finite(x)]
  if (length(x) < 3) return(NA_real_)
  cor(x[-length(x)], x[-1], use = "complete.obs")
}

stats <- cycles %>%
  group_by(country, iso2, group, component) %>%
  summarise(
    observations = sum(is.finite(cycle)),
    cycle_sd = sd(cycle, na.rm = TRUE),
    cycle_mean_abs = mean(abs(cycle), na.rm = TRUE),
    persistence_ar1 = safe_cor_lag1(cycle),
    skewness = moments::skewness(cycle, na.rm = TRUE),
    kurtosis = moments::kurtosis(cycle, na.rm = TRUE),
    mean_yoy_growth = mean(yoy_growth, na.rm = TRUE),
    sd_yoy_growth = sd(yoy_growth, na.rm = TRUE),
    .groups = "drop"
  )
```

```

# Compare each component with GDP over exactly the same country-quarter sample.
# This avoids mistaking different start dates for differences in volatility.
gdp_reference <- cycles %>%
  filter(component == "GDP") %>%
  select(country, date, gdp_cycle = cycle)

component_comparison <- cycles %>%
  inner_join(gdp_reference, by = c("country", "date")) %>%
  group_by(country, component) %>%
  summarise(
    comparison_observations = sum(is.finite(cycle) & is.finite(gdp_cycle)),
    relative_volatility_to_gdp = sd(cycle, na.rm = TRUE) /
      sd(gdp_cycle, na.rm = TRUE),
    corr_with_domestic_gdp = cor(cycle, gdp_cycle, use = "complete.obs"),
    .groups = "drop"
  )

stats <- stats %>%
  left_join(component_comparison, by = c("country", "component"))

stats %>% arrange(country, component)

```

```

# A tibble: 35 × 15
  country iso2 group      component observations cycle_sd cycle_mean_abs
  <chr>   <chr> <chr>      <chr>          <int>    <dbl>      <dbl>
1 Australia AU   Asia-Pacific Consumption    201     1.64       1.04
2 Australia AU   Asia-Pacific GDP          201     1.28       0.882
3 Australia AU   Asia-Pacific Investment    201     4.17       3.25
4 China    CN   Asia-Pacific GDP          140     1.42       0.854
5 India    IN   Asia-Pacific Consumption    116     3.37       1.71
6 India    IN   Asia-Pacific GDP          119     3.00       1.58
7 India    IN   Asia-Pacific Investment    116     5.98       3.11
8 Indonesia ID   Focus      Consumption    101     1.28       0.782
9 Indonesia ID   Focus      GDP            156     2.50       1.41
10 Indonesia ID   Focus      Investment     101     2.75       2.15
# i 25 more rows
# i 8 more variables: persistence_ar1 <dbl>, skewness <dbl>, kurtosis <dbl>,
# mean_yoy_growth <dbl>, sd_yoy_growth <dbl>, comparison_observations <int>,
# relative_volatility_to_gdp <dbl>, corr_with_domestic_gdp <dbl>

```

Correlation with Indonesia's cycle

```

safe_cor <- function(x, y) {
  paired <- is.finite(x) & is.finite(y)
  if (sum(paired) < 3) return(NA_real_)
  cor(x[paired], y[paired])
}

```

```

}

indonesia_cycles <- cycles %>%
  filter(country == "Indonesia") %>%
  select(date, component, indonesia_cycle = cycle)

correlations <- cycles %>%
  inner_join(indonesia_cycles, by = c("date", "component")) %>%
  group_by(country, iso2, group, component) %>%
  summarise(
    corr_with_indonesia = safe_cor(cycle, indonesia_cycle),
    correlation_observations = sum(is.finite(cycle) &
is.finite(indonesia_cycle)),
    .groups = "drop"
  ) %>%
  arrange(component, desc(corr_with_indonesia))

correlations

```

```

# A tibble: 35 × 6
  country    iso2 group component corr_with_indonesia correlation_observat...1
  <chr>      <chr> <chr> <chr>          <dbl>          <int>
1 Indonesia  ID    Focus Consumpt...      1            101
2 Philippines PH    ASEAN Consumpt...  0.843           99
3 Malaysia  MY    ASEAN Consumpt...  0.751           37
4 Singapore  SG    ASEAN Consumpt...  0.718          100
5 Australia  AU    Asia... Consumpt...  0.610          101
6 Thailand   TH    ASEAN Consumpt...  0.541           89
7 Japan      JP    Asia... Consumpt...  0.516          101
8 India      IN    Asia... Consumpt...  0.508          101
9 United Stat... US    Comp... Consumpt...  0.324          101
10 South Korea KR    Asia... Consumpt...  0.324          101
# i 25 more rows
# i abbreviated name: 1correlation_observations

```

```

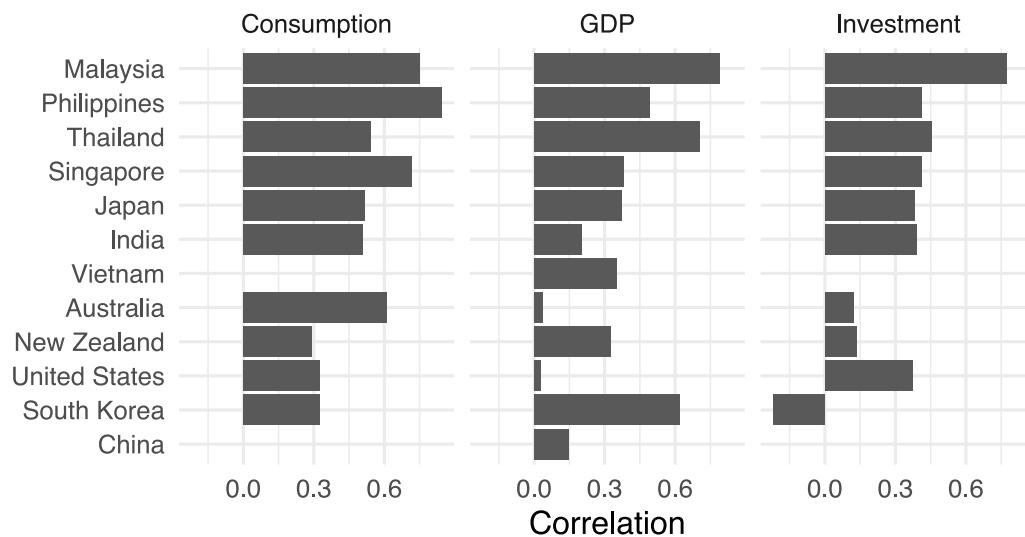
correlations %>%
  filter(country != "Indonesia") %>%
  ggplot(aes(x = reorder(country, corr_with_indonesia), y =
corr_with_indonesia)) +
  geom_col() +
  coord_flip() +
  facet_wrap(~ component) +
  labs(
    title = "Business-cycle correlation with Indonesia",
    subtitle = "Contemporaneous correlation of HP-filtered real component
cycles",

```

```
x = NULL,
y = "Correlation"
)
```

Business-cycle correlation with Indonesia

Contemporaneous correlation of HP-filtered real component c



7. Rolling synchronisation

A single full-sample correlation can hide structural change. The next calculation uses a 20-quarter (5-year) rolling correlation with Indonesia.

```
roll_cor <- function(x, y, width = 20) {
  zoo::rollapplyr(
    cbind(x, y),
    width = width,
    by.column = FALSE,
    fill = NA_real_,
    FUN = function(z) {
      paired <- complete.cases(z)
      if (sum(paired) < 3) return(NA_real_)
      cor(z[paired, 1], z[paired, 2])
    }
  )
}

rolling <- cycles %>%
  filter(component == "GDP") %>%
  select(date, country, cycle) %>%
  pivot_wider(names_from = country, values_from = cycle) %>%
  arrange(date)
```

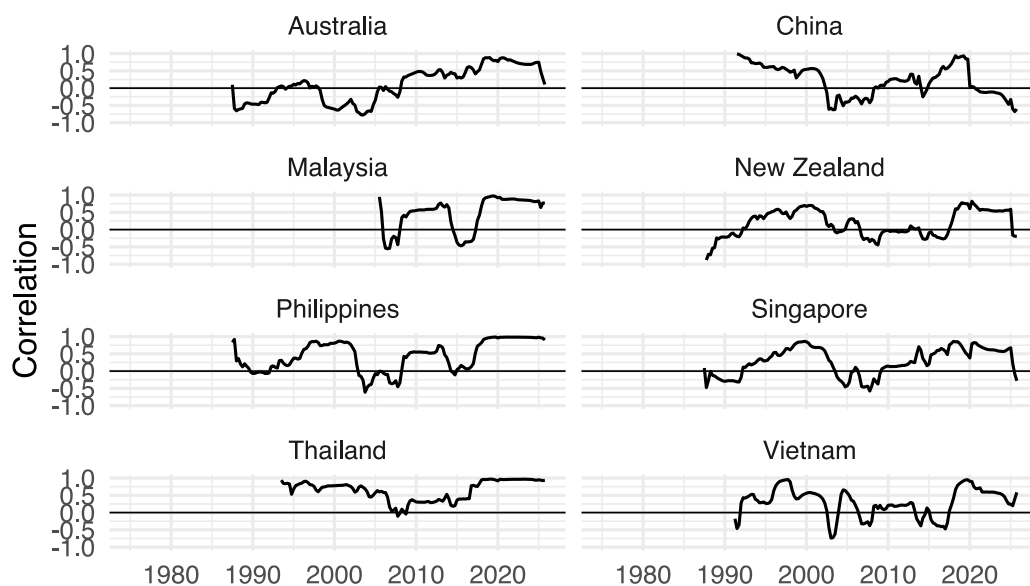
```

rolling_long <- map_dfr(setdiff(names(rolling), c("date", "Indonesia")),
  function(ctry) {
    tibble(
      date = rolling$date,
      component = "GDP",
      country = ctry,
      rolling_corr = roll_cor(rolling[[ctry]], rolling$Indonesia, 20)
    )
  })

rolling_long %>%
  filter(country %in% c(
    "Malaysia", "Thailand", "Philippines", "Singapore", "Vietnam",
    "Australia", "New Zealand", "China"
  )) %>%
  ggplot(aes(date, rolling_corr, group = country)) +
  geom_hline(yintercept = 0, linewidth = 0.3) +
  geom_line() +
  facet_wrap(~ country, ncol = 2) +
  coord_cartesian(ylim = c(-1, 1)) +
  labs(
    title = "Five-year rolling business-cycle correlation with Indonesia",
    x = NULL,
    y = "Correlation"
  )

```

Five-year rolling business-cycle correlation with Indones



8. Export data for a D3 visualisation

```
out_dir <- "d3_data"
dir.create(out_dir, showWarnings = FALSE)

cycles %>%
  select(date, country, iso2, group, component, cycle, yoy_growth,
qoq_annualised) %>%
  write_csv(file.path(out_dir, "business_cycle_data.csv"))

stats %>%
  left_join(
    correlations %>% select(country, component, corr_with_indonesia,
correlation_observations),
    by = c("country", "component")
  ) %>%
  write_csv(file.path(out_dir, "business_cycle_stats.csv"))

rolling_long %>%
  write_csv(file.path(out_dir, "rolling_correlations.csv"))
```